Chapter 28

Supporting External Exception Review

This section describes how FT PharmaSuite supports the handling of exceptions by an external system.

The maintenance and review of exceptions is not limited to FT PharmaSuite. If specific exceptions should be documented and reviewed with an external QMS, they can be sent to the external QMS for being processed.
In the default configuration, the feature is disabled. It can be enabled with the **LibraryHolder/services-S88-impl.jar/ActivateExternalExceptionHandling** configuration key.

In case the feature is enabled, FT PharmaSuite and the external QMS perform the following steps when an exception is created:

- FT PharmaSuite checks if the exception fulfills the criterion to be handled externally: is the risk class of the exception set to **High**.
  In the default configuration, this is true. The required risk class can be configured with the **LibraryHolder/services-S88-impl.jar/MinimalRiskClassForExternalExceptionHandling** configuration key.
- If the criterion is fulfilled, FT PharmaSuite creates a communication identifier (unique exception ID) and stores it in the exception.
  By default, the identifier consists of the order or workflow identifier concatenated by '-' and the exception identifier.
- Then, FT PharmaSuite sends a message with the exception data to the external QMS, sets the communication status of the exception to **Sent**, and the status of the exception to **In review**.
- When the external QMS has received and processed the message, it sends a reply message to FT PharmaSuite with
  - the communication identifier,
  - an identifier created by the external QMS (if applicable),
  - an updated exception status (if applicable, e.g., **Closed**),
  - an updated communication status (if applicable, e.g., **Received**), and
  - optional additional textual information.

- An FT PharmaSuite listener process that runs on the transition server receives the message from the external QMS.
  It retrieves the corresponding exception using the communication identifier and checks if the related order or workflow is not already in the **Reviewed**, **Canceled**, or **Annulled** statuses. If not, FT PharmaSuite creates an exception comment using the received data and adds it to the exception.
  FT PharmaSuite can process more than one reply received for an exception.

Besides the basic pattern, exceptions will also be sent to an external QMS in the following cases:

- When a comment is added to an exception, the risk class of the exception can also be changed.
  As described above, FT PharmaSuite checks if the risk class now fulfills the criterion. If so, a message is sent to the external QMS.
- In case the external exception handling is enabled, exceptions that have not yet been sent to the external QMS can be sent manually, depending on the communication status of the exception.
  In the default configuration, the communication status must be **Not sent** or **Error**. The required status can be configured with the **LibraryHolder/services-S88-impl.jar/CommunicationStatesToAllowManualExceptionSending** configuration key.
  In the Production Response Client, in the Details panel of the exception dashboard, select the exceptions to be sent and click the **Send exception** button. FT PharmaSuite requests an electronic signature prior to sending the exceptions and adds a corresponding comment to each exception. It sends the message for each exception as described above.

Adapting the External Exception Handling

The external exception handling is implemented in the *IS88ExternalExceptionHandlingService* service. To adapt the behavior, implement your own service derived from *IS88ExternalExceptionHandlingService*. The service provides the methods to send exceptions to and handle reply messages from the external QMS. The methods are called by FT PharmaSuite.

- *boolean sendExceptions(Map<IMESExceptionRecord, IS88ExceptionRecordContext>)*
  is called when one or more exceptions were created with one signature.
- *boolean sendException(IMESExceptionRecord, IS88ExceptionRecordContext)*
  is called when an exception was created or the risk class of an exception was changed by adding a comment.

- *boolean sendExceptionsManually(List<IMESExceptionRecord>, String, IESignatureExecutor)*
  is called when the **Send exception** button in the Details panel of the exception dashboard was clicked.
- *void receiveException(String, String, Status, IMESChoiceElement, String)*
  is called when a reply message from the external QMS was received.

In addition, the service provides further methods:

- *String createCommunicationIdentifier(IMESExceptionRecord, IS88ExceptionRecordContext)*
  creates the communication identifier that is used to identify an exception when a reply is received from the external QMS. By default, the identifier consists of the order or workflow identifier concatenated by '-' and the exception identifier. The communication identifier must be unique per exception.
- *boolean isHandledExternally(IMESExceptionRecord, IS88ExceptionRecordContext)*
  determines if an exception can be handled by an external QMS. By default, this is true if the external exception handling is enabled, and the risk class-specific criterion is fulfilled.
- *Map<String, Object> getOtherInformation(IMESExceptionRecord, IS88ExceptionRecordContext)*
  can be implemented/overridden by a customized service implementation in order to provide additional information which is transferred to the external QMS. The standard implementation returns an empty map, i.e., no other information is passed to the external QMS.

TIP
Please note that the returned values of the *Object* type are converted into strings with their *toString()* method and then embedded in the XML document that is sent to the external QMS.

Configuring the UI of the Exception Dashboard

FT PharmaSuite provides the following configuration options for the Details panel of the exception dashboard in the Production Response Client:

- To display the **Communication status** and **External ID** columns in the exceptions list, set the **LibraryHolder/services-S88-impl.jar/ActivateExternalExceptionHandling** configuration key to **True**.
- To hide the **CAPA ID** box in the exception recording panel and the **CAPA ID** column in the exception list, set the **LibraryHolder/apps-exceptiondashboard-impl.jar/HideCapaID** configuration key to **True**.
- To display the **Classification** attribute in the exceptions list and the exception recording panel and to activate the required business logic, set the **LibraryHolder/services-S88-impl.jar/ActivateExceptionClassification** configuration key to **True**.
  The setting also applies to the Production Execution Client.

Connecting with an External Exception System

The interface between FT PharmaSuite and an external QMS is an XML document sent by JMS messages via ActiveMQ.

In the standard configuration of an FT PharmaSuite installation, the message broker URL to be used for the communication is **ssl://<host name>:61647**.

The corresponding XML schema is defined in *RAExternalExceptionHandling.xsd* and provided with the *ftps-b2mml-v0600-<PharmaSuite version>.jar*. The schema is compliant to the S88/S95 standards and based on the B2MML standard schema.

The schema definition contains two top-level XML elements:

- Exception message: ProcessRecordedException (page 441)
- Response message: ConfirmRecordedException (page 444)

The messages in both directions are text messages containing XML-formatted strings.

TIP

The communication between FT PharmaSuite and an external QMS is asynchronous, i.e., FT PharmaSuite does not actively wait for a reply.

General Structure of the XML Messages

In general, each XML message consists of three containers:

- **ApplicationArea** contains the sender and the receiver of a message.
  The name of the sender is configured with the **LibraryHolder/services-S88-ifc.jar/ExternalExceptionHandlingSender** configuration key. Its default value is MES.
  The name of the receiver is configured with the **LibraryHolder/services-S88-ifc.jar/ExternalExceptionHandlingReceiver** configuration key. Its default value is QMS.
- **DataArea** contains the actual information that needs to be exchanged.
- **OtherInformations** is a container that can be used to transfer additional data that is needed and not contained in the standard. Optionally, it contains a list of additional key/value pairs (see section "Extending Messages" (page 445)).

Message: ProcessRecordedException

The message to inform the external QMS about an exception is an XML document of the **ProcessRecordedException** type. The exception data is located in the **DataArea** container (page 441).

Example of a 'ProcessRecordedException' message of a workflow exception

```
<ProcessRecordedException
    xmlns="http://www.mesa.org/xml/B2MML-V0600"
    xmlns:b2m="http://www.mesa.org/xml/B2MML-V0600-AllExtensions">
  <ApplicationArea>
    <Sender>
      <LogicalID>MES</LogicalID>
    </Sender>
    <Receiver>
      <LogicalID>QMS</LogicalID>
    </Receiver>
  </ApplicationArea>
  <DataArea>
    <PSUniqueExceptionID>WF1700000731 - 002</PSUniqueExceptionID>
    <OrderOrWorkflowIdentifier>WF1700000731</OrderOrWorkflowIdentifier>
    <ExceptionIdentifier>002</ExceptionIdentifier>
    <ExceptionDescription>This is the exception text</ExceptionDescription>
    <ExceptionReference>WF1700000731/Unit Procedure-01/dgOp01-01/
                        EquipBindTestPhase-01</ExceptionReference>
    <ExceptionRiskChoiceElement>
      <b2m:Meaning>Low</b2m:Meaning>
      <b2m:Localized>Low</b2m:Localized>
      <b2m:Value>10</b2m:Value>
      <b2m:ChoiceListName>RiskClass</b2m:ChoiceListName>
    </ExceptionRiskChoiceElement>
    <ExceptionCategoryChoiceElement>
      <b2m:Meaning>User defined</b2m:Meaning>
      <b2m:Localized>User-defined</b2m:Localized>
      <b2m:Value>10</b2m:Value>
      <b2m:ChoiceListName>ExceptionCategory</b2m:ChoiceListName>
    </ExceptionCategoryChoiceElement>
    <ExceptionClassificationChoiceElement>
```

```
      <b2m:Meaning>Classification 20</b2m:Meaning>
      <b2m:Localized>Classification 2</b2m:Localized>
      <b2m:Value>20</b2m:Value>
      <b2m:ChoiceListName>ExceptionClassification</b2m:ChoiceListName>
    </ExceptionClassificationChoiceElement>
    <ExceptionFirstSignature>
      <TimeStamp>2017-12-01T13:40:33.957+01:00</TimeStamp>
      <Description>Exception Record added</Description>
      <RecordReference>700196</RecordReference>
      <Name>dgerr</Name>
      <Name>Detamana Gerr</Name>
      <ChangeIndication>c3171348779a6cda4c5b5c1982c66ad7</ChangeIndication>
      <Reason>Confirmation</Reason>
    </ExceptionFirstSignature>
    <StationIdentifier>STA_dg_D1</StationIdentifier>
    <WorkCenterIdentifier>WKC_dg_D</WorkCenterIdentifier>
  </DataArea>
</ProcessRecordedException>
```

XML schema of the message (defined in RAExternalExceptionHandling.xsd)

```
<xsd:complexType name="ProcessRecordedExceptionType">
  <xsd:annotation>
    <xsd:documentation>
      Message to inform an external system about an exception that was recorded
      in PharmaSuite.
      See also S95 Part 5:  Chapter 5.4 PROCESS verb
    </xsd:documentation>
  </xsd:annotation>
  <xsd:sequence>
    <xsd:element name="ApplicationArea" type="TransApplicationAreaType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="DataArea" type="ProcessRecordedException_DataAreaType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="OtherInformations" type="OtherInformationType"
                 minOccurs="0" maxOccurs="1" />
  </xsd:sequence>
  <xsd:attribute name="schemaVersionID" type="xsd:normalizedString" fixed="1.0" />
</xsd:complexType>
<xsd:complexType name="ProcessRecordedException_DataAreaType">
  <xsd:annotation>
    <xsd:documentation>
      Data area for 'ProcessRecordedExceptionType'.
      Remarks: "TargetBatchIdentifier" and "TargetMaterialNumber" are optional since
               they will be empty for exceptions related to workflows.
               "StationIdentifier" is optional since it can be empty if the exception
               is recorded by PharmaSuite's Exception Dashboard.
      See also S95 Part 5: Chapter 4.2 Message structure
    </xsd:documentation>
  </xsd:annotation>
  <xsd:sequence>
    <xsd:element name="PSUniqueExceptionID" type="IdentifierType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="OrderOrWorkflowIdentifier" type="IdentifierType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="TargetBatchIdentifier" type="IdentifierType"
                 minOccurs="0" maxOccurs="1" />
    <xsd:element name="TargetMaterialNumber" type="IdentifierType"
                 minOccurs="0" maxOccurs="1" />
    <xsd:element name="ExceptionIdentifier" type="IdentifierType"
```

```
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionDescription" type="TextType"
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionReference" type="TextType"
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionRiskChoiceElement"
                type="BatchInfoExtended:ChoiceElementType"
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionCategoryChoiceElement"
                type="BatchInfoExtended:ChoiceElementType"
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionClassificationChoiceElement"
                type="BatchInfoExtended:ChoiceElementType"
                minOccurs="0" maxOccurs="1" />
    <xsd:element name="ExceptionFirstSignature"
                type="PersonnelIdentificationManifestType"
                minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionSecondSignature"
                type="PersonnelIdentificationManifestType"
                minOccurs="0" maxOccurs="1" />
    <xsd:element name="StationIdentifier" type="IdentifierType"
                minOccurs="0" maxOccurs="1" />
    <xsd:element name="WorkCenterIdentifier" type="IdentifierType"
                minOccurs="1" maxOccurs="1" />
  </xsd:sequence>
</xsd:complexType>
```

The signatures of an exception (i.e., the *ExceptionFirstSignature* element and the optional *ExceptionSecondSignature* element) are represented by the B2MML standard *PersonnelIdentificationManifestType* type. The conventions to pass on the signer information within a signature element are:

- The FT PharmaSuite user name of the signer is always stored in the first <*Name*> element.
- The full name of the signer is always stored in the second <*Name*> element.
- The optionally available email address of a signer is always stored in the third <*Name*> element. If no email address is available, the third <*Name*> element is omitted.

Message: ConfirmRecordedException

The response to a **ProcessRecordedException** message received by FT PharmaSuite is an XML document of the **ConfirmRecordedException** type. The relevant data is contained in the **DataArea** container (page 441).

Example of a 'ConfirmRecordedException' message to set the exception status to "To be closed"

```
<ConfirmRecordedException xmlns="http://www.mesa.org/xml/B2MML-V0600">
  <ApplicationArea>
    <Sender>
      <LogicalID>QMS</LogicalID>
    </Sender>
    <Receiver>
      <LogicalID>MES</LogicalID>
    </Receiver>
  </ApplicationArea>
  <DataArea>
    <PSUniqueExceptionID>WF1700000731 - 002</PSUniqueExceptionID>
    <ExceptionCommunicationStatusMeaning>Received</ExceptionCommunicationStatusMeaning>
    <ExternalExceptionID>WF1700000731 - 002 - 2</ExternalExceptionID>
    <ExceptionStatusMeaning>To be closed</ExceptionStatusMeaning>
  </DataArea>
</ConfirmRecordedException>
```

XML schema of the message (defined in RAExternalExceptionHandling.xsd)

```
<xsd:complexType name="ConfirmRecordedExceptionType">
  <xsd:annotation>
    <xsd:documentation>
      Message to inform PharmaSuite about an exception that was processed by an
      external system.
      It is assumed that this exception was previously sent by PharmaSuite.
      See also S95 Part 5: Chapter 5.8 CONFIRM verb
    </xsd:documentation>
  </xsd:annotation>
  <xsd:sequence>
    <xsd:element name="ApplicationArea" type="TransApplicationAreaType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="DataArea" type="ConfirmRecordedException_DataAreaType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="OtherInformations" type="OtherInformationType"
                 minOccurs="0" maxOccurs="1" />
  </xsd:sequence>
  <xsd:attribute name="schemaVersionID" type="xsd:normalizedString" fixed="1.0" />
</xsd:complexType>
<xsd:complexType name="ConfirmRecordedException_DataAreaType">
  <xsd:annotation>
    <xsd:documentation>
      Data area for 'ProcessExceptionUpdateType'.
      Remarks: "ExceptionCommunicationStatusMeaning" contains the new communication status
               of the exception in form of the meaning of the corresponding choice element
               of PharmaSuite's choice list "ExceptionCommunicationStatus".
               "ExternalExceptionID" is optional. If it is present but empty an already
               existing external exception ID in the exception will be deleted.
               "ExceptionStatusMeaning" is optional and contains the exception status in
               form of the meaning of the corresponding choice element of PharmaSuite's
```

```
                choice list "ExceptionStatus". If present the status of the exception is
                updated accordingly when this message is processed.
                "ExternalExceptionComment" is optional. If present its contents will be
                appended to the description of the exception comment that is created upon
                receipt of this message.
      See also S95 Part 5: Chapter 4.2 Message structure
    </xsd:documentation>
  </xsd:annotation>
  <xsd:sequence>
    <xsd:element name="PSUniqueExceptionID" type="IdentifierType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExceptionCommunicationStatusMeaning" type="IdentifierType"
                 minOccurs="1" maxOccurs="1" />
    <xsd:element name="ExternalExceptionID" type="IdentifierType"
                 minOccurs="0" maxOccurs="1" />
    <xsd:element name="ExceptionStatusMeaning" type="IdentifierType"
                 minOccurs="0" maxOccurs="1" />
    <xsd:element name="ExternalExceptionComment" type="TextType"
                 minOccurs="0" maxOccurs="1" />
  </xsd:sequence>
</xsd:complexType>
```

Receiving Messages from FT PharmaSuite and Sending Responses to FT PharmaSuite

The EXCEPTION_RECORD_EXTERNAL_RESPONSE message queue (i.e., point-to-point communication) is used to exchange messages between FT PharmaSuite and an external QMS. Thus, the external QMS should listen for messages and send replies using this queue.

The response must be an ActiveMQ text message with an appropriate **ConfirmRecordedExceptionType** XML document (page 444) as the actual content.

An external QMS can send more than one response to a single **ProcessRecordedException** message received from FT PharmaSuite.

Extending Messages

If there is a need to transfer further data that is not contained in the default messages, it is possible to add a list of key/value pairs to the XML document for both directions. The additional data for the **ProcessRecordedException** message is automatically obtained by calling *IS88ExternalExceptionHandlingService.getOtherInformation()* method (see section "Adapting the External Exception Handling" (page 438)).

In the XML document, the additional data is stored in the **OtherInformations** container (page 441).

Example of a 'ProcessRecordedException' message with 'OtherInformations'

```
<ProcessRecordedException
    xmlns="http://www.mesa.org/xml/B2MML-V0600"
    xmlns:b2m="http://www.mesa.org/xml/B2MML-V0600-AllExtensions">
  <ApplicationArea>
    <Sender>
      <LogicalID>MES</LogicalID>
    </Sender>
    <Receiver>
      <LogicalID>QMS</LogicalID>
    </Receiver>
  </ApplicationArea>
  <DataArea>
    <PSUniqueExceptionID>E1-1512137344167 - 001</PSUniqueExceptionID>
    <!-- ... -->
    <!—- further details omitted -->
    <!-- ... -->
  </DataArea>
  <OtherInformations>
    <ID>DecimalValue</ID>
    <ID>IntegerValue</ID>
    <ID>StringValue</ID>
    <Value>
      <ValueString name="DecimalValue">4.23</ValueString>
    </Value>
    <Value>
      <ValueString name="IntegerValue">12</ValueString>
    </Value>
    <Value>
      <ValueString name="StringValue">This is a test string</ValueString>
    </Value>
  </OtherInformations>
</ProcessRecordedException>
```

The receiving QMS must know the data types and the semantics of the key/value pairs. In particular it has to know how the string representation of the values have to be interpreted and converted back into their native data types.

The keys appear twice per key/value pair: as <*ID*> element (without value) and as *name* attribute of the corresponding <*ValueString*> element. Basically, the receiving QMS has two possibilities to parse the content of the **OtherInformations** container:

- Iterate over the <*ID*> elements and thereby pick up the nth <*ValueString*> element, where n is the number of the current <*ID*> element. The order of <*ID*> elements and <*ValueString*> elements corresponds 1:1.
- Iterate only over the <*ValueString*> elements and obtain the keys with their *name* attribute.